Literature notes: tamper-evident logging, Merkle trees, blockchain anchoring, agent auditing

Project: Auditable AI Agents: An Immutable On-Chain Audit Trail for LLM Agent Actions
Author: Manikandan Sankaranarayanan
Status: Month 1 first pass. **Citation details below were written from memory and have not been checked against the papers.** Open each source and confirm authors, venue, pages and year before using it in the thesis.

---

1. Why these readings

The project makes three claims that need support from existing work: (a) logs can be made tamper-evident with hash chains and Merkle trees; (b) a public ledger can serve as an external witness so that even the log owner cannot rewrite history; (c) LLM agents need accountable records of what they did. The readings below cover each.

---

2. Recommended base paper

S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in Proc. 18th USENIX Security Symposium, 2009.

Why it is the base paper:

- It is the standard reference for building a tamper-evident log from a Merkle-tree structure, with efficient proofs that an entry is in the log and that a later version of the log extends an earlier one.
- Its threat model (an untrusted logger who may rewrite history, and clients who check commitments) is the same problem this project solves for agent logs.
- It leaves room for the contribution here: it commits to the log through signed commitments held by auditors, whereas this project anchors commitments on a public blockchain, and applies the idea to LLM agent actions with measured cost.

What to extract when reading: the history-tree definition, how membership and consistency proofs work, the performance numbers, and the assumptions about who holds the commitments.

---

3. Core background

Topic	Source (verify before citing)	What it gives this project
Hash-chained secure audit logs	B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," ACM Trans. Information and System Security, 1999	The original idea of chaining log entries so that later tampering is detectable; the limits of a chain without an external witness

Topic	Source (verify before citing)	What it gives this project
Merkle trees	R. C. Merkle, "A digital signature based on a conventional encryption function," CRYPTO 1987	Folding many hashes into one root; inclusion proofs
Timestamping with hash links	S. Haber and W. S. Stornetta, "How to time-stamp a digital document," Journal of Cryptology, 1991	Linking documents by hash to prove they existed in a given order; ancestor of anchoring
Transparency logs	B. Laurie, A. Langley and E. Kasper, "Certificate Transparency," RFC 6962, 2013	A production Merkle-tree log with signed tree heads; the source of the leaf and node domain-separation prefixes (0x00 and 0x01) used here
Blockchain	S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008	Append-only ledger that no single party controls
Smart contracts	G. Wood, "Ethereum: A secure decentralised generalised transaction ledger" (Yellow Paper)	Programs on a ledger; the gas cost model behind the cost measurements
SHA-256	NIST, FIPS 180-4, Secure Hash Standard	The hash function used in Python and in Solidity
LLM agents	S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," ICLR 2023	The reason-act-observe loop that the demo agent follows and that the audit log records
AI governance	NIST, AI Risk Management Framework (AI RMF 1.0), 2023	Why accountability and traceability records for AI systems are wanted

---

4. What the existing work does not cover (the gap)

1. Classic tamper-evident logging assumes ordinary software logs. Agent logs have a particular shape (prompt, model response, tool call, tool result) and a particular worry (an agent or its operator rewriting what the agent did).
2. Blockchain-logging papers often put whole logs on-chain, which is costly and exposes data. Here only a Merkle root per batch goes on-chain, and the cost-versus-delay trade-off is measured.
3. The extra protection a ledger gives over a signed or chained log alone, namely detection of a **consistent ly rewritten** log, is shown directly by test in this project (see `docs/TESTING.md`).

---

5. Position of this project

An engineering study that applies Merkle-tree tamper-evident logging with public-ledger anchoring to LLM agent actions, proves detection of edit, deletion, insertion, reordering, truncation and consistent rewrite by automated test, and measures the gas, latency and scaling cost.

It does **not** claim: that the agent's outputs are correct, that a compromised logger cannot record falsehoods before anchoring, or that the prototype is production-ready.

---

6. Reading plan

Week	Reading
1	Crosby and Wallach (full paper); RFC 6962 sections 2 and 3
2	Schneier and Kelsey; Haber and Stornetta
3	ReAct; NIST AI RMF sections on accountability
4	Two or three recent papers on blockchain-anchored logging and on LLM-agent observability (search needed; none are cited here because none have been verified)

---

7. Open questions

- Should a consistency proof (the log at time 2 extends the log at time 1) be added, as in Crosby and Wallach, or are per-batch roots enough for the thesis scope?
- Should entries be signed by the agent's key as well as hashed (identity, not just integrity)?
- Is a public testnet run worth the added complexity, or is the local chain enough for the evaluation?